Cómo funciona el banco

Programar y depurar microcontroladores a distancia con una BeagleBoard-xM

Banco de programación y depuración remota · octubre de 2026

Repositorio: `https://github.com/ElianUlrich/STM32-bench`

> **En pocas palabras.** Una BeagleBoard-xM queda en el banco del laboratorio, conectada por USB a los microcontroladores (STM32, ESP32, Arduino) y a sus programadores. Desde el navegador de cualquier PC conectada a la misma red privada (Tailscale) se sube el firmware, se graba, se depura línea por línea y se lee el puerto serie, sin estar físicamente frente al banco.

Contenido

1. La idea
2. Las piezas del sistema
3. Qué pasa cuando tocás un botón
4. Cómo se programa cada familia de microcontroladores
5. Cómo se evita grabar el micro equivocado
6. Depuración
7. Monitor serie, historial y estado de la placa
8. Seguridad
9. Organización del software
10. Desarrollo sin hardware y pruebas
11. Límites actuales

1. La idea

Cuando el hardware está en un laboratorio y quien programa está en otro lugar, cada cambio de firmware exige ir hasta el banco. La solución es dejar en el banco una computadora chica (la BeagleBoard-xM, una placa con Linux) que hace de intermediaria: tiene enchufados los programadores y los microcontroladores, y ofrece una aplicación web para usarlos.

Necesidad	Cómo la resuelve el banco
Grabar firmware a distancia	Subís el .elf/.hex/.bin a la web y tocás Flashear.
Depurar (pausar, ver variables, paso a paso)	Pestaña Depurar en la web, o GDB/VS Code/CubeIDE conectados al servidor GDB del banco.
Ver lo que el micro manda por serie	Monitor serie en la web, con envío de texto.
Que nadie de afuera pueda entrar	Acceso sólo por Tailscale (red privada cifrada) y login.
Que funcione sin pantalla ni teclado	La placa arranca sola el servicio; todo se maneja desde la web o por SSH.

2. Las piezas del sistema

La figura muestra los bloques. A la izquierda la PC del usuario; en el medio el túnel de Tailscale; a la derecha la BeagleBoard con los programas que corren en ella y el hardware conectado a sus puertos USB.

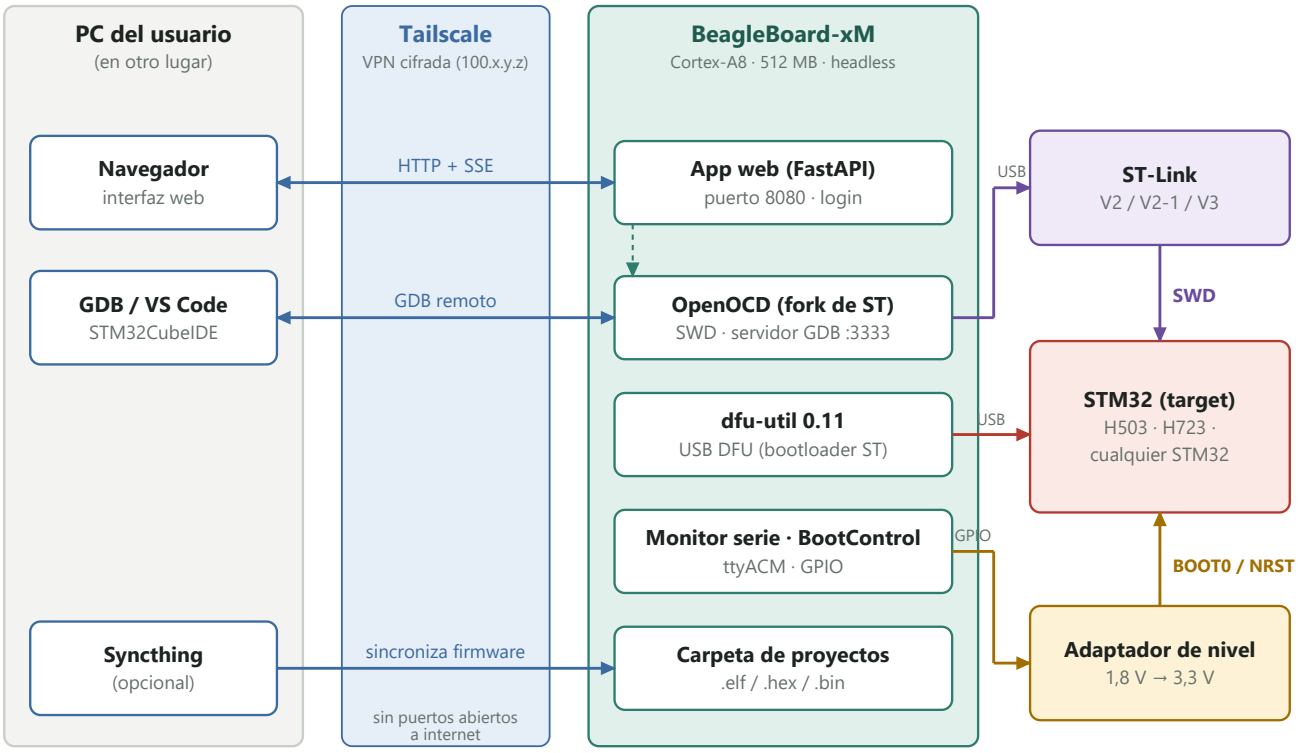


Figura 1 — Bloques del sistema (ejemplo con un STM32 por ST-Link y USB DFU).

- **PC**: sólo necesita un navegador y Tailscale. Opcionalmente PowerShell para entrar a la terminal de la placa, y un depurador (GDB, VS Code, CubeIDE).
- **Tailscale**: red privada (VPN) que conecta la PC con la placa aunque estén en redes distintas. Cada equipo recibe una IP fija del tipo 100.x.y.z. No hay que abrir puertos en el router.
- **BeagleBoard-xM**: placa con procesador ARM Cortex-A8 y Linux (Ubuntu). Corre la aplicación web y las herramientas de programación.

- **Herramientas**: OpenOCD (fork de ST) para ST-Link, dfu-util para el bootloader USB de ST, esptool para ESP32/ESP8266, avrdude para Arduino, y GDB para depurar.
- **Programadores y placas**: ST-Link (SWD) al STM32; el USB de las placas ESP32 y Arduino; opcionalmente un adaptador USB-serie para el monitor.

3. Qué pasa cuando tocás un botón

Todas las acciones siguen el mismo camino. Ejemplo: grabar un programa en un STM32.

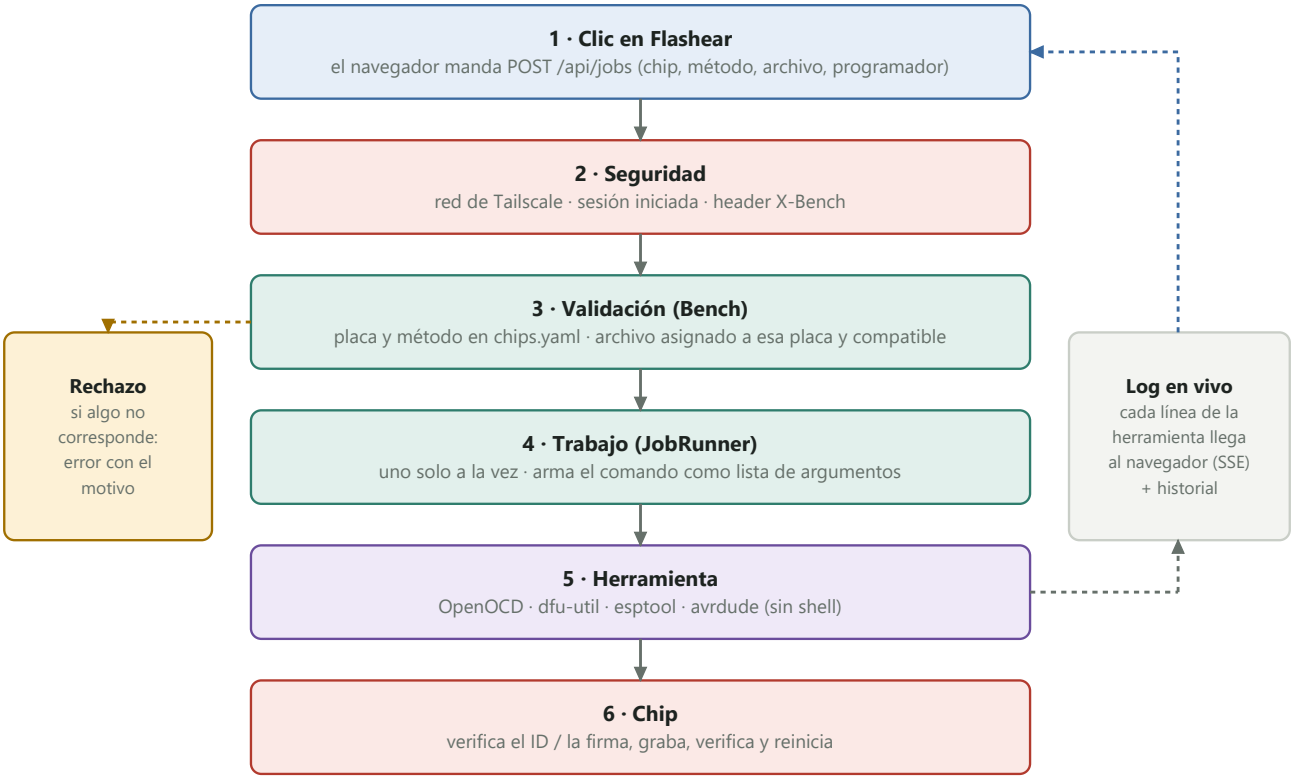


Figura 2 — Recorrido de un pedido, desde el navegador hasta el chip y de vuelta.

- El navegador nunca manda comandos: manda *qué* quiere (placa, acción, archivo). El servidor decide *cómo* y arma el comando de la herramienta.
- Hay **un solo trabajo a la vez**: si el programador está ocupado, el pedido se rechaza en lugar de encolarse. Un trabajo largo se puede cancelar.
- Cada línea que escribe la herramienta aparece en el **log en vivo** (Server-Sent Events) y el resultado queda en el **historial**.

4. Cómo se programa cada familia

Cada placa del catálogo (`config/chips.yaml`, 28 placas) declara uno o más métodos. La interfaz sólo muestra los botones que ese método soporta y explica por qué los demás están deshabilitados.

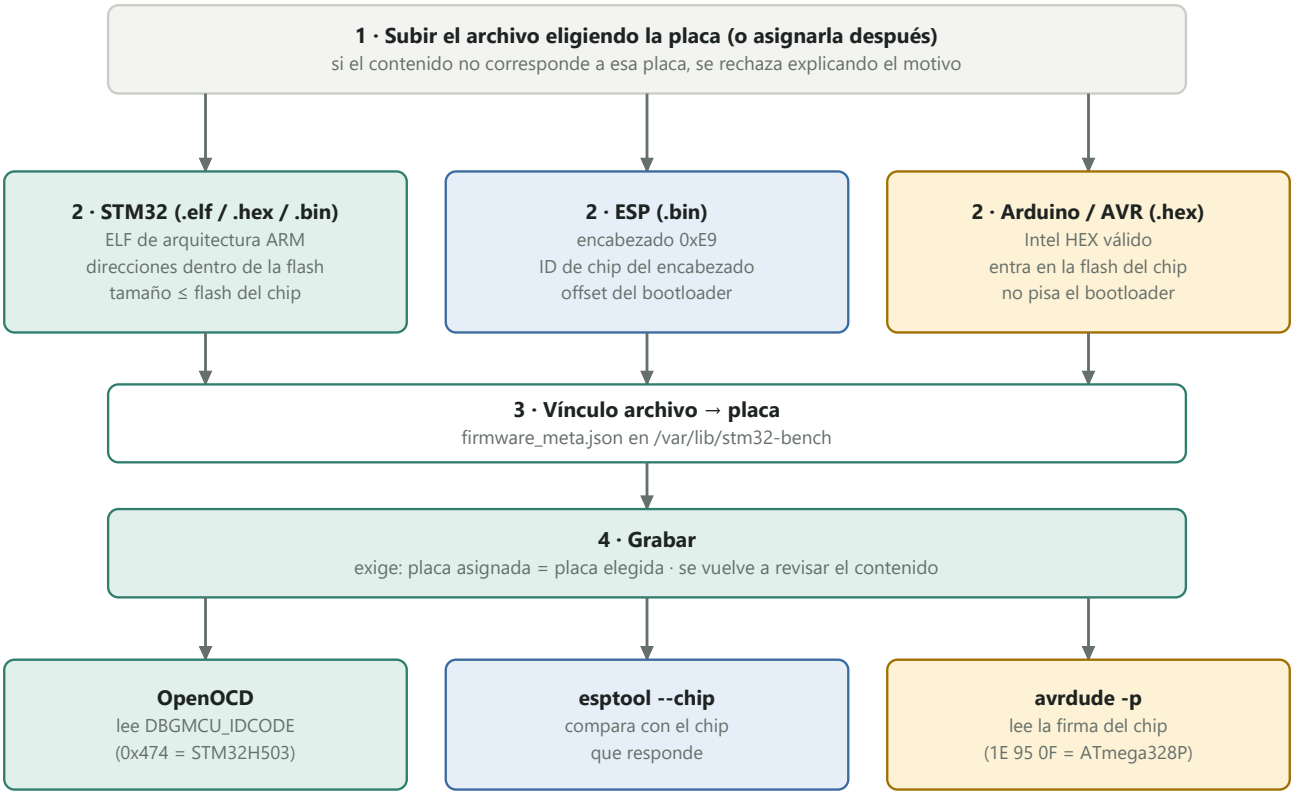
Método	Placas	Conexión	Acciones
ST-Link (SWD) · OpenOCD	STM32 (H5, H7, F1, F4, L4, G0, G4…)	ST-Link al conector SWD (SWDIO, SWCLK, GND)	grabar, verificar, borrar, reset, halt, run, identificar, depurar
USB DFU · dfu-util	STM32 con bootloader USB	USB del propio STM32 (PA11/PA12), con BOOT0 en alto	grabar, verificar, borrar

Método	Placas	Conexión	Acciones
Serie · esptool	ESP8266, ESP32, S2, S3, C2, C3, C5, C6, C61, H2, P4	USB de la placa (conversor o USB-JTAG)	grabar, verificar, borrar, reset, identificar
Serie · avrdude	Uno, Nano, Mega, Leonardo, Micro, Pro Mini…	USB del Arduino (o USBasp por ISP)	grabar, verificar, reset, identificar (borrar sólo por ISP)

- **STM32 por SWD**: OpenOCD habla con el ST-Link. Antes de grabar lee el registro de identificación del chip (DBGMCU_IDCODE) y aborta si no es el esperado.
- **STM32 por USB DFU**: el chip entra a su bootloader de fábrica (BOOT0 en alto + reset). La app convierte el .elf/.hex a .bin y lo graba con dfu-util. No permite depurar.
- **ESP32**: esptool pone el chip en modo descarga solo (líneas DTR/RTS del USB). La app reconoce si el .bin es sólo la aplicación (va en 0x10000) o la imagen combinada (va en 0x0).
- **Arduino**: avrdude habla con el bootloader por el puerto serie. En Leonardo/Micro la app abre el puerto a 1200 baudios para que el chip salte al bootloader.

5. Cómo se evita grabar el micro equivocado

Con muchas placas en el mismo banco, grabar un binario en la placa que no corresponde es fácil y puede dañar hardware. Por eso cada archivo se sube diciendo para qué placa es, y hay tres controles:



5 · Antes de escribir, la herramienta confirma el chip conectado; si no coincide, aborta sin tocar la flash

Figura 3 — Las tres capas: contenido del archivo, asignación y chip conectado.

Ejemplo. Un binario compilado para ESP32-S3 trae ese dato en su encabezado: si lo asignás a un ESP32 común, la web lo rechaza al subirlo. Si igual hubiera otro chip conectado, esptool lo detecta antes de escribir y aborta.

6. Depuración

Depurar es ejecutar el programa bajo control: frenarlo en una línea (breakpoint), avanzar de a una línea, ver y cambiar variables, y ver la pila de llamadas. Sirve para encontrar por qué el programa hace algo distinto de lo esperado, sin llenarlo de printf.

Pestaña Depurar (todo en la web)

La aplicación corre GDB en la placa, lo conecta a OpenOCD y muestra el resultado. Se ve el código fuente, se marcan breakpoints con un clic en el número de línea y se usan los mismos atajos que en los IDE: F5 continuar, F6 pausar, F10 paso, F11 entrar, Shift+F11 salir.

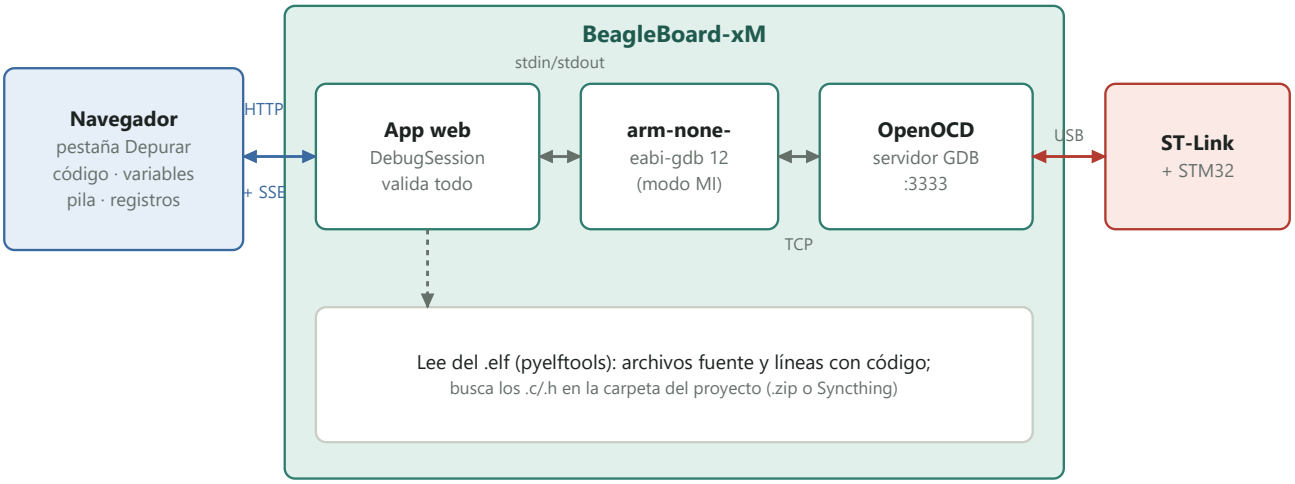


Figura 4 — La pestaña Depurar: la app maneja GDB en la placa.

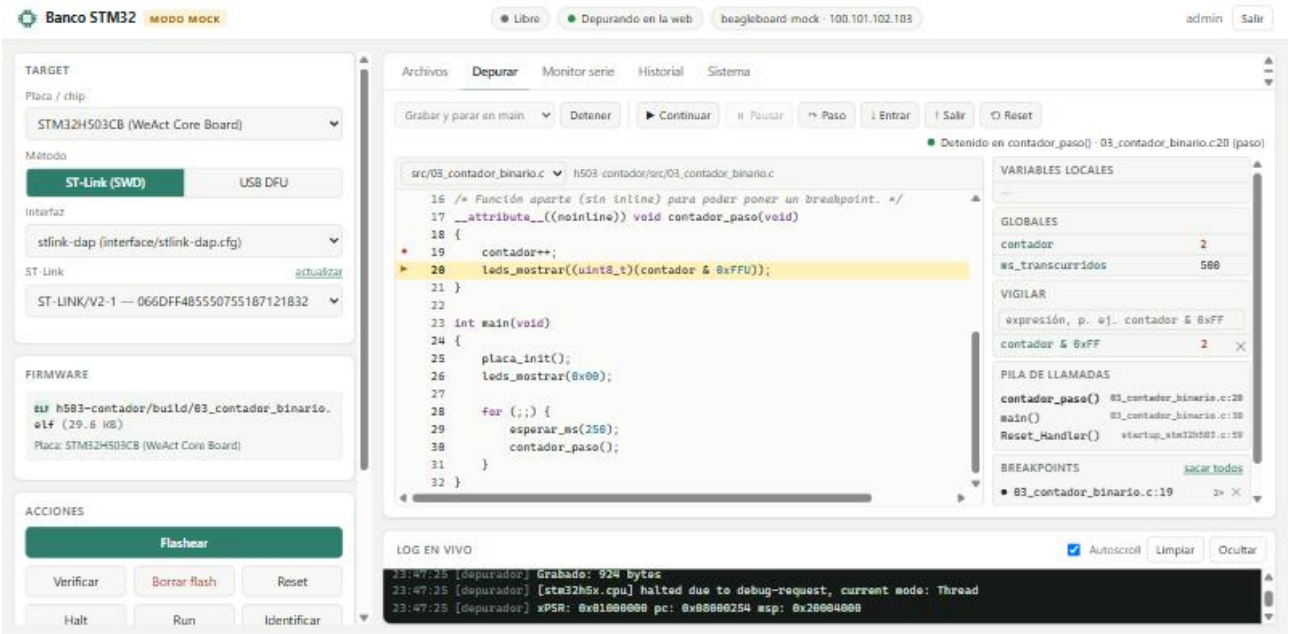


Figura 5 — Pestaña Depurar detenida en un breakpoint (modo simulación).

- Para ver el código hacen falta las fuentes: se sube el proyecto en .zip (el .elf y la carpeta con los .c/.h). La app busca cada archivo usando las rutas que guarda el .elf.
- Tres formas de empezar: grabar y parar en main, reiniciar y parar en main, o conectarse sin reiniciar (frena el programa donde esté).
- Pausar detiene todo el microcontrolador: en un sistema de audio o de control, mientras está frenado, la salida se corta.

Servidor GDB (para usar tu propio depurador)

También se puede iniciar sólo el servidor GDB y conectarse desde la PC con arm-none-eabi-gdb, VS Code (Cortex-Debug) o STM32CubeIDE a `100.99.235.81:3333`. OpenOCD acepta un único cliente: o la web, o un depurador externo.

7. Monitor serie, historial y estado de la placa

- **Monitor serie**: lista los puertos USB-serie de la placa (/dev/ttyUSB*, /dev/ttyACM*), con velocidad configurable, y permite enviar texto con el fin de línea elegido. Si se graba una placa por el mismo puerto, el monitor se cierra mientras graba y se vuelve a abrir solo.
- **RTT**: el "printf" del programa viaja por el mismo ST-Link (SWD), sin adaptador USB-serie ni pines extra y sin frenar el micro. El programa escribe en un buffer en RAM (bloque `_SEGGER_RTT`) y OpenOCD lo lee mientras corre; en el monitor serie aparece como el puerto "RTT — por el ST-Link". Funciona también mientras se depura.
- **USB**: en Sistema se ve todo lo que está enchufado (útil para diagnosticar, por ejemplo, si un STM32 en modo DFU aparece como 0483:df11). Con un hub compatible con uhubctl se puede apagar, prender o reiniciar la energía de cada puerto; el puerto que lleva a la red nunca se corta.
- **Historial**: cada operación con su resultado, duración y archivo.
- **Sistema**: IP, tiempo encendida, memoria, disco, versiones de las herramientas, y los botones Apagar y Reiniciar (con confirmación). Después de apagar hay que esperar unos 30 segundos antes de cortar la alimentación; la placa no se puede volver a encender a distancia.

8. Seguridad

La aplicación puede ejecutar programas en la placa y grabar hardware, así que se protegió en capas:

Capa	Qué hace
Red	Sólo se aceptan conexiones desde la red de Tailscale (100.64.0.0/10) o desde la propia placa. No se abren puertos en el router.
Login	Usuario y contraseña (guardada como hash, nunca en texto). Cookie de sesión firmada que vence. Límite de intentos fallidos.
Pedidos	Toda acción exige un header especial (protección contra sitios ajenos).
Comandos	Nunca se arma un comando con texto del usuario: listas de argumentos, sin shell. Placas e interfaces sólo de los archivos de configuración; nombres de archivo con caracteres permitidos; rutas siempre dentro de la carpeta de proyectos.
Privilegios	El servicio corre como un usuario sin privilegios. sudo sólo le permite apagar y reiniciar.
Depurador	El puerto GDB escucha sólo en la IP de Tailscale. La web nunca le manda texto libre a GDB; las expresiones se validan.

9. Organización del software

El servidor está escrito en Python (FastAPI) y la interfaz en HTML, CSS y JavaScript sin dependencias externas, para que funcione liviano en la BeagleBoard.

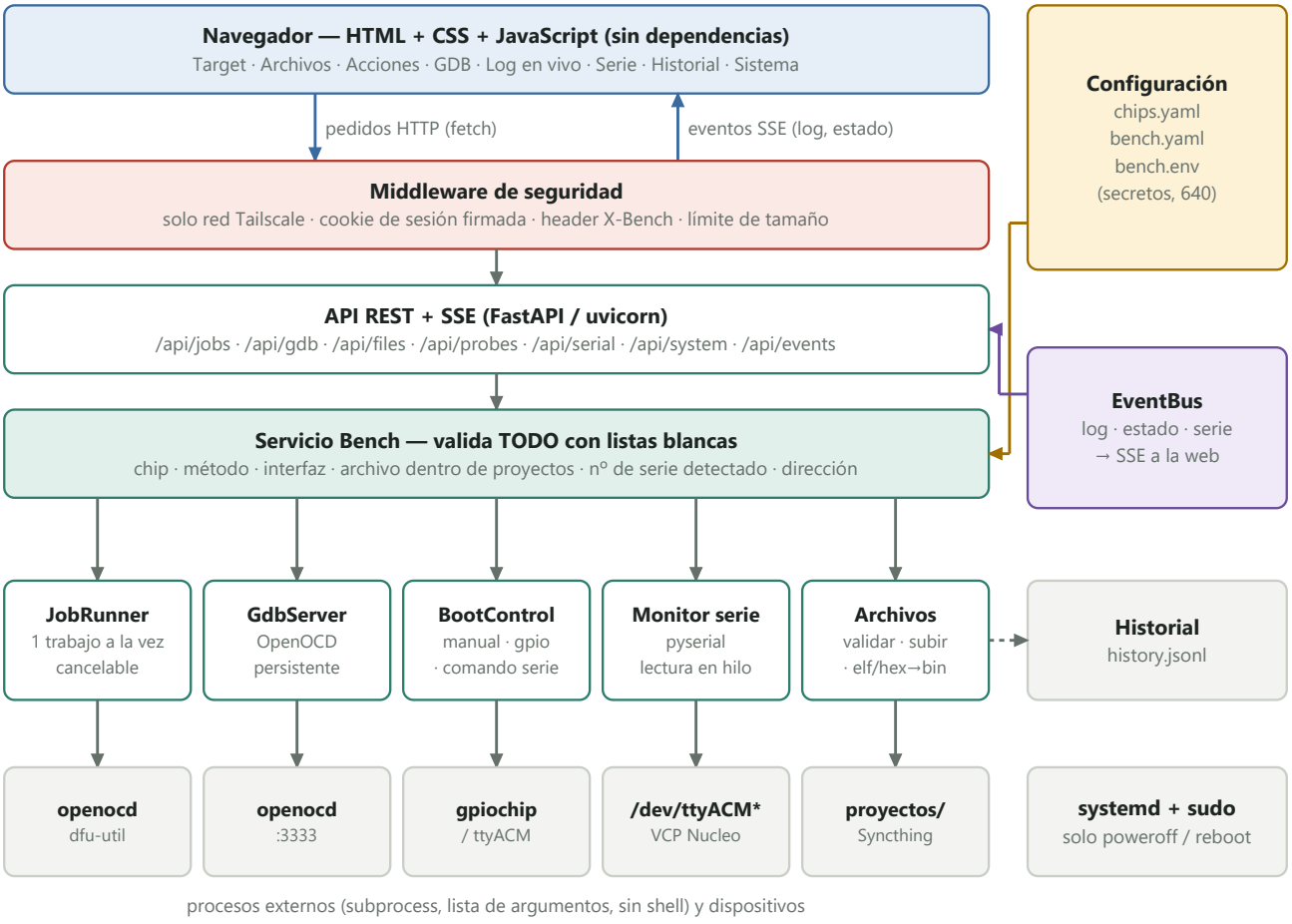


Figura 6 — Capas de software dentro de la placa.

Carpeta / archivo	Qué contiene
app/	servidor: API, validaciones, trabajos, depurador, monitor serie, sistema
web/	la interfaz (index.html, app.js, style.css)
config/	chips.yaml (placas), bench.yaml (herramientas, red, límites)
scripts/	instalación, compilación de OpenOCD y GDB, simuladores para el modo mock
deploy/	servicio systemd, reglas udev, permisos de sudo
firmware/h503_leds/	programas de prueba para el STM32H503 (LEDs y puerto serie)
tests/	pruebas automáticas (pytest)
docs/	esta guía, la de instalación y el documento de trabajo con la bitácora

10. Desarrollo sin hardware y pruebas

Con la variable `BENCH_MOCK=1` la aplicación corre en la PC y reemplaza OpenOCD, dfu-util, esptool, avrdude y GDB por simuladores que se ejecutan como programas reales. Así se desarrolla y se prueba toda la interfaz sin placa. El simulador de GDB ejecuta el contador binario real, con sus líneas y funciones.

Las pruebas automáticas (más de 220) cubren validación de archivos, armado de comandos, configuración, vínculo archivo-placa, el protocolo de GDB, la subida de proyectos .zip y sesiones de depuración completas.

11. Límites actuales

- La depuración en la web es sólo para STM32 por SWD. Todavía no hay vista de memoria, watchpoints, breakpoints condicionales ni periféricos por nombre.
- El USB DFU del STM32H503 todavía no fue probado con éxito en la placa (no enumera; posible cable sólo de carga).
- ESP32 y Arduino están probados en simulación; falta probarlos con placas reales.
- La BeagleBoard-xM tiene USB con poca corriente: varias placas a la vez requieren un hub con fuente.